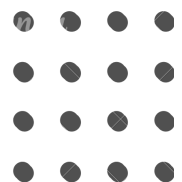




VMES Grinds





Leetcode: 2347 Best Poker Hand





2347. Best Poker Hand

Solved

Easy

Topics

Companies

Hint

You are given an integer array `rank` and a character array `suit`. You have 5 cards where the i^{th} card has a rank of `rank[i]` and a suit of `suit[i]`.

The following are the types of **poker hands** you can make from best to worst:

1. "Flush": Five cards of the same suit.
2. "Three of a Kind": Three cards of the same rank.
3. "Pair": Two cards of the same rank.
4. "High Card": Any single card.

Return a string representing the **best** type of **poker hand** you can make with the given cards.

Note that the return values are **case-sensitive**.



Example 1:

Input: `ranks = [13,2,3,1,9], suits = ["a","a","a","a","a"]`

Output: `"Flush"`

Explanation: The hand with all the cards consists of 5 cards with the same suit, so we have a "Flush".

Example 2:

Input: `ranks = [4,4,2,4,4], suits = ["d","a","a","b","c"]`

Output: `"Three of a Kind"`

Explanation: The hand with the first, second, and fourth card consists of 3 cards with the same rank, so we have a "Three of a Kind".

Note that we could also make a "Pair" hand but "Three of a Kind" is a better hand.

Also note that other cards could be used to make the "Three of a Kind" hand.

Example 3:

Input: `ranks = [10,10,2,12,9], suits = ["a","b","c","a","d"]`

Output: `"Pair"`

Explanation: The hand with the first and second card consists of 2 cards with the same rank, so we have a "Pair".

Note that we cannot make a "Flush" or a "Three of a Kind".



Constraints:

- `ranks.length == suits.length == 5`
- `1 <= ranks[i] <= 13`
- `'a' <= suits[i] <= 'd'`
- No two cards have the same rank and suit.



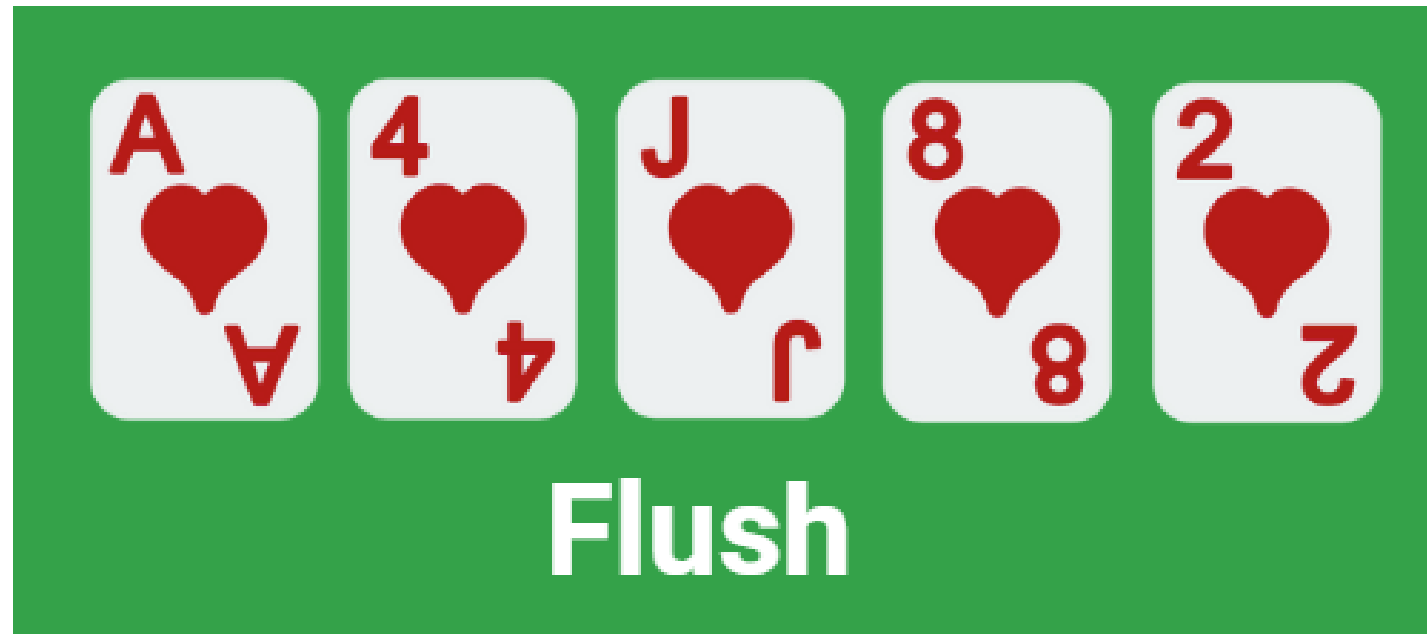
Explanation





VMES Grinds

Five cards with the same suit (kind)



Three cards with the same value



Two cards with the same value



One Pair

No cards have equal value (not also same suits)





- 1.Count the ranks (values) using hashtable**
- 2.Count the suits using hashtable**
- 3.Check for Flush, Three of a kind, Two of a kind
or just high card preferable in that order!**
- 4.Done!**



Initial Approach





- 1.Count the ranks!
- 2.Count the suits!
- 3.Check the cards!
- 4.Print the best card!

```
class Solution:
    def bestHand(self, ranks, suits):
        rank_count = {}
        suit_count = {}

        for i in range(5):
            r = ranks[i]
            s = suits[i]

            if r in rank_count:
                rank_count[r] += 1
            else:
                rank_count[r] = 1

            if s in suit_count:
                suit_count[s] += 1
            else:
                suit_count[s] = 1

        # 1. Check Flush
        for s in suit_count:
            if suit_count[s] == 5:
                return "Flush"

        # 2. Check Three of a Kind
        for r in rank_count:
            if rank_count[r] >= 3:
                return "Three of a Kind"

        # 3. Check Pair
        for r in rank_count:
            if rank_count[r] == 2:
                return "Pair"

        # 4. High Card
        return "High Card"
```

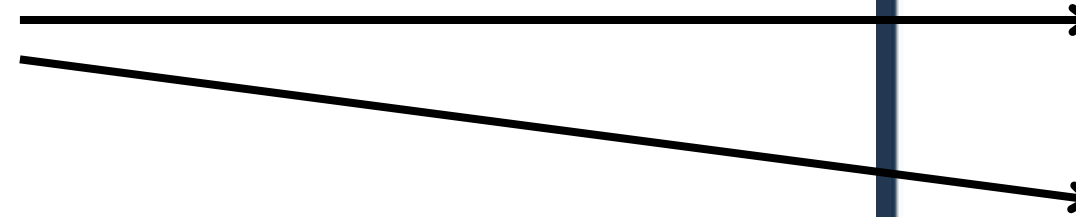


Optimized Approach





Early Stopping aka Pruning!



```
class Solution:
    def bestHand(self, ranks, suits):
        rank_count = {}
        suit_count = {}

        for i in range(5):
            r = ranks[i]
            s = suits[i]

            if r in rank_count:
                rank_count[r] += 1
            else:
                rank_count[r] = 1

            if s in suit_count:
                suit_count[s] += 1
            else:
                suit_count[s] = 1

            # Early flush detection
            if suit_count[s] == 5:
                return "Flush"

            # Early three-of-a-kind detection
            if rank_count[r] == 3:
                return "Three of a Kind"

        # Check for pair
        for r in rank_count:
            if rank_count[r] == 2:
                return "Pair"

        return "High Card"
```



Time and Space Complexity Analysis





```
class Solution:
    def bestHand(self, ranks, suits):
        rank_count = {}
        suit_count = {}

        for i in range(5):
            r = ranks[i]
            s = suits[i]

            if r in rank_count:
                rank_count[r] += 1
            else:
                rank_count[r] = 1

            if s in suit_count:
                suit_count[s] += 1
            else:
                suit_count[s] = 1

        # 1. Check Flush
        for s in suit_count:
            if suit_count[s] == 5:
                return "Flush"

        # 2. Check Three of a Kind
        for r in rank_count:
            if rank_count[r] >= 3:
                return "Three of a Kind"

        # 3. Check Pair
        for r in rank_count:
            if rank_count[r] == 2:
                return "Pair"

        # 4. High Card
        return "High Card"
```

Space - $O(1)$

Time - $O(1)$



```
class Solution:
    def bestHand(self, ranks, suits):
        rank_count = {}
        suit_count = {}

        for i in range(5):
            r = ranks[i]
            s = suits[i]

            if r in rank_count:
                rank_count[r] += 1
            else:
                rank_count[r] = 1

            if s in suit_count:
                suit_count[s] += 1
            else:
                suit_count[s] = 1

            # Early flush detection
            if suit_count[s] == 5:
                return "Flush"

            # Early three-of-a-kind detection
            if rank_count[r] == 3:
                return "Three of a Kind"

        # Check for pair
        for r in rank_count:
            if rank_count[r] == 2:
                return "Pair"

        return "High Card"
```

Space - $O(1)$

Time - $O(1)$



Key Takeaway



Counting Problem =
Hashtable!

Early Stopping where
relevant!

Consider other DS such as
Set!